AI-Based Multi-Cloud Migration Decision Framework for Healthcare Enterprises using Multi-Criteria Decision Intelligence

Complete Team Development Guide — BCSE355L Cloud Architecture Design

Charles (24BIT0522) · Tanmay Kapoor (24BIT0079) · Rani Gupta (24BIT0543)

Contents

0. How to Use This Document **3**

1. Project Overview **4**
1.1 Title . . . 4
1.2 Problem We Are Solving . . . 4
1.3 Our Solution, In One Paragraph . . . 4
1.4 The Three Layers (map directly to the three teammates) . . . 4
1.5 Why This Design Is Novel . . . 5

2. Full System Architecture **6**
2.1 End-to-End Data Flow . . . 6
2.2 Repository Structure (everyone works in this one repo) . . . 6
2.3 Git Branching Strategy . . . 7

3. CHARLES — Prediction Layer (AI for Network Forecasting) **8**
3.1 What This Module Does . . . 8
3.2 Responsibilities Checklist . . . 8
3.3 Data Schema (agree on this — everyone else depends on it) . . . 8
3.4 Step-by-Step Build Plan . . . 8
3.5 AI Coding Prompt — Data Generator . . . 9
3.6 AI Coding Prompt — Preprocessing . . . 9
3.7 AI Coding Prompt — Model + Training . . . 10
3.8 AI Coding Prompt — Serving Function (what the rest of the team calls) . . . 10
3.9 Definition of Done (Charles) . . . 11

4. TANMAY — Decision Layer (MCDM / AHP Engine) **12**
4.1 What This Module Does . . . 12
4.2 Responsibilities Checklist . . . 12
4.3 The AHP Method, In Plain Terms (for your own understanding, and for the report) . . . 12
4.4 Data Contract (what this module receives and returns) . . . 12
4.5 AI Coding Prompt — AHP Core Logic . . . 13
4.6 AI Coding Prompt — Router + Override Logic . . . 13
4.7 AI Coding Prompt — Unit Tests . . . 14
4.8 Definition of Done (Tanmay) . . . 15

5. RANI — Execution Layer (Infrastructure, Security, Storage, Dashboard) **16**
5.1 What This Module Does . . . 16

5.2 Responsibilities Checklist . . . 16
5.3 Data Contract . . . 16
5.4 AI Coding Prompt — Storage Layer . . . 16
5.5 AI Coding Prompt — Auth Layer . . . 17
5.6 AI Coding Prompt — Logger . . . 17
5.7 AI Coding Prompt — Edge Node & Cloud Node . . . 18
5.8 AI Coding Prompt — Dashboard . . . 19
5.9 The HIPAA / Security Narrative (write this into the report, Rani's section) . . . 19
5.10 Definition of Done (Rani) . . . 20

6. HOW WE COMBINE EVERYTHING (Integration Layer) 21
6.1 The Full Call Chain . . . 21
6.2 AI Coding Prompt — The Integration API . . . 21
6.3 AI Coding Prompt — End-to-End Simulation Runner . . . 22
6.4 Integration Testing Checklist (Rani leads, everyone participates) . . . 23

7. AWS Migration Path (For Later — Not Needed Now) 24

8. Full Development Timeline 25

9. Requirements File 26

10. Final Pre-Submission Checklist 27

0. How to Use This Document

This document is the **single source of truth** for building the full project — every teammate should read their own section end-to-end, and everyone should read Section 6 (Integration) so we all agree on how the pieces connect.

How to use the code prompts in this guide: every module below includes a ready-to-use **AI coding prompt**. Copy the prompt exactly into Antigravity, VS Code Copilot Chat, or Cursor, in the correct file/folder as instructed, and let it generate the implementation. Do not skip the "Context to paste first" blocks — they tell the AI tool what the rest of the system looks like so the generated code stays compatible with everyone else's modules.

We are building the **full system**, not a cut-down demo. Real AWS deployment happens later (Section 7); everything here is written so that shifting to AWS later requires changing configuration, not rewriting logic.

1. Project Overview

1.1 Title

AI-Based Multi-Cloud Migration Decision Framework for Healthcare Enterprises using Multi-Criteria Decision Intelligence (working name: *AI-Based Serverless Telemedicine Workload Offloader*)

1.2 Problem We Are Solving

Telemedicine and Internet of Medical Things (IoMT) devices — ECG monitors, wearables, ICU patient sensors — generate continuous, high-volume data streams that need ultra-low-latency handling. Hospitals must decide, for every incoming data packet, whether to process it:

- **locally at the hospital edge** (fast, safe, but limited compute), or
- **in the centralized cloud** (scalable, cheaper per-unit, but subject to network latency and higher risk if the connection degrades).

Existing systems get this wrong in three ways:

1. **Static & reactive routing** — all data goes to the cloud by default; systems only respond *after* a network failure has already caused delay, with no predictive capability.
2. **Cost-blind routing** — routine and life-critical data are treated identically, wasting cloud spend on data that could safely wait.
3. **Compliance risk** — Protected Health Information (PHI) travels continuously over public networks, creating HIPAA exposure during congestion, and monolithic architectures don't scale to sudden concurrent sensor bursts.

1.3 Our Solution, In One Paragraph

We build a system that **predicts** network degradation before it happens, **mathematically decides** (using Multi-Criteria Decision Intelligence / AHP) whether each piece of data should go to the edge or the cloud — weighing patient criticality, predicted latency, and compute cost together — and **executes** that decision through a secure hybrid edge-cloud pipeline that keeps critical PHI local during vulnerable periods. During a clinical emergency, the system can autonomously override cost-optimized routing and force critical data to the edge, with zero human intervention.

1.4 The Three Layers (map directly to the three teammates)

Layer	Owner	One-line job
Prediction Layer	Charles	Forecast whether the network is about to degrade
Decision Layer	Tanmay	Given the forecast + patient criticality + cost, decide: Edge or Cloud?
Execution & Security Layer	Rani	Actually route the data, log everything, enforce access control and PHI protection

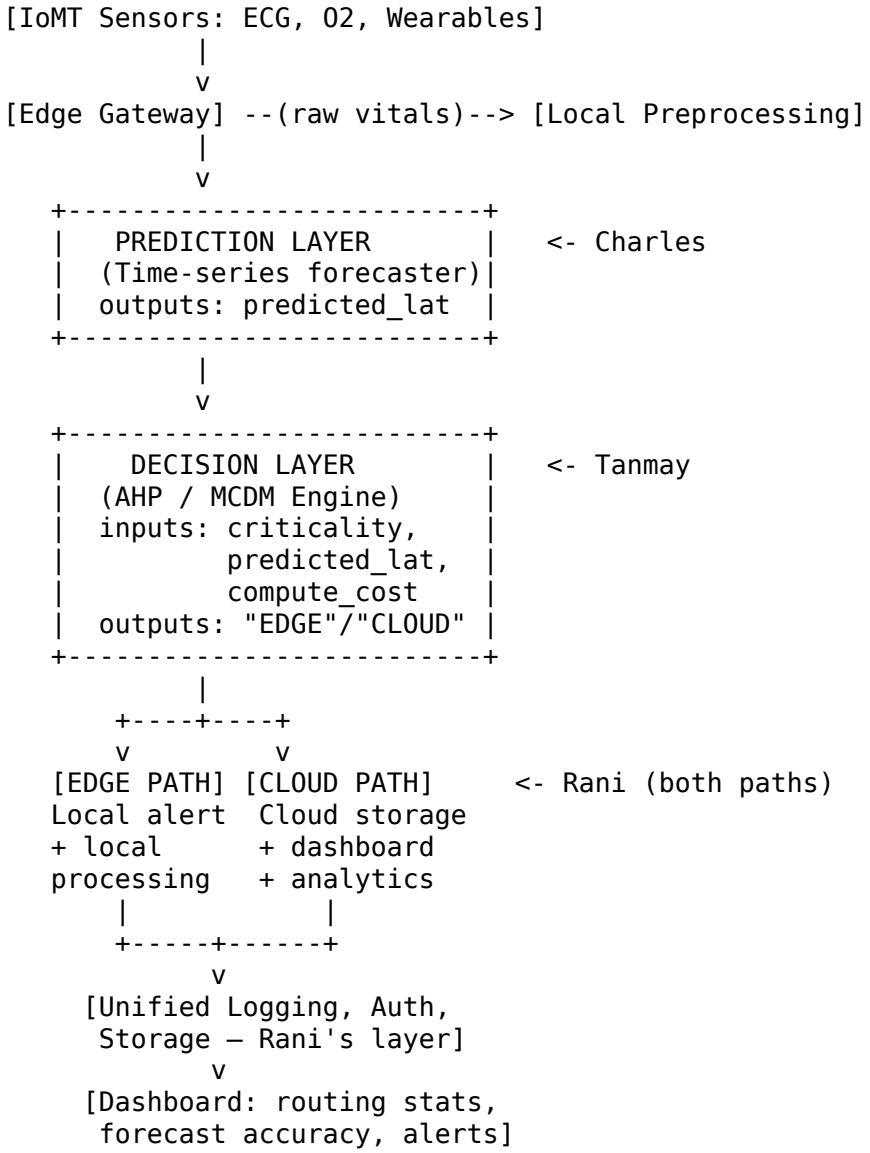
1.5 Why This Design Is Novel

Existing approaches (see literature survey) use **binary threshold rules** ("if latency > 100ms, switch"). We use:

- **Prediction instead of reaction** — forecasting is built into the routing decision itself.
- **Multiple criteria instead of one** — AHP balances criticality, latency, and cost together, not one metric in isolation.
- **Transparent math instead of a black box** — AHP is fully explainable, which matters for clinical/regulatory audits, unlike opaque Deep Reinforcement Learning approaches used in prior work.
- **Fully event-driven / serverless target architecture** — no EC2, no Kubernetes; everything is designed to run as small, independently scalable functions.

2. Full System Architecture

2.1 End-to-End Data Flow



2.2 Repository Structure (everyone works in this one repo)

```
telemedicine-cloud-project/
├── src/
│   ├── prediction/           # Charles
│   │   ├── data_generator.py
│   │   ├── preprocessing.py
│   │   ├── model.py
│   │   ├── train.py
│   │   └── predict_service.py
│   ├── decision/             # Tanmay
│   │   ├── ahp.py
│   │   ├── router.py
```

```
│   │   └── override_rules.py
│   ├── execution/              # Rani
│   │   ├── edge_node.py
│   │   ├── cloud_node.py
│   │   ├── auth.py
│   │   ├── storage.py
│   │   └── logger.py
│   └── api/                    # shared integration layer
│       └── main.py
├── dashboard/                  # Streamlit dashboard (shared, Rani leads)
│   └── app.py
├── data/
│   ├── raw/
│   └── processed/
├── notebooks/
│   └── train_model.ipynb       # Colab-linked
├── tests/
│   ├── test_prediction.py
│   ├── test_decision.py
│   └── test_execution.py
├── docs/
│   ├── architecture_diagrams/
│   └── research/
├── requirements.txt
└── README.md
```

2.3 Git Branching Strategy

```
main
 ^
 |
develop
 ├── feature/prediction   (Charles)
 ├── feature/decision     (Tanmay)
 └── feature/execution    (Rani)
```

Rules: - Nobody commits directly to `main` or `develop`. - Each person commits to their own `feature/*` branch, opens a Pull Request into `develop`. - At least one other teammate reviews every PR before merge. - Once `develop` is stable and integration-tested, merge into `main` and tag a release (e.g., `v1.0-phase1`).

3. CHARLES — Prediction Layer (AI for Network Forecasting)

3.1 What This Module Does

Given a rolling window of recent network telemetry (bandwidth, packet loss, jitter, signal strength) from IoMT devices, predict whether the network is about to degrade in the next N seconds, and output a **predicted latency value** that the Decision Layer (Tanmay) will consume.

3.2 Responsibilities Checklist

- ☐ Generate a realistic synthetic dataset of hospital network + IoMT telemetry
- ☐ Preprocess data: normalize, create sliding windows for time-series input
- ☐ Build and train an LSTM (or GRU) time-series forecasting model
- ☐ Evaluate model accuracy (MAE/RMSE on held-out data)
- ☐ Wrap the trained model in a `predict()` function with a clean interface
- ☐ Expose it as a small service/function the rest of the system can call
- ☐ Produce accuracy graphs (predicted vs actual latency) for the report

3.3 Data Schema (agree on this — everyone else depends on it)

Each telemetry record (JSON):

```
{
  "timestamp": "2026-09-22T10:15:32Z",
  "device_id": "ECG-101",
  "bandwidth_mbps": 42.3,
  "packet_loss_pct": 1.2,
  "jitter_ms": 8.5,
  "signal_strength_dbm": -67,
  "current_latency_ms": 34.0
}
```

Model output (what Charles's module hands to Tanmay's module):

```
{
  "device_id": "ECG-101",
  "predicted_latency_ms": 210.5,
  "network_degradation_risk": "HIGH",
  "forecast_horizon_seconds": 30
}
```

3.4 Step-by-Step Build Plan

1. **Synthetic data generation** — simulate normal traffic patterns plus injected "degradation events" (sudden latency spikes, packet loss bursts) so the model has something real to learn.
2. **Preprocessing** — Min-Max scale all numeric fields; build sliding windows of e.g. 20 past readings → predict the next reading.
3. **Model** — start with a simple 2-layer LSTM; this is enough for the coursework and trains fast in Colab (CPU/GPU).
4. **Training** — 80/20 train-test split, track MAE/RMSE, save the trained model as `model.h5` or `model.pkl`.

5. **Serving** — wrap in a plain Python function `predict_latency(window: list[dict]) -> dict` so Tanmay's router can call it directly, with **no AWS dependency** for now (this becomes a SageMaker endpoint call later, same input/output shape).

3.5 AI Coding Prompt — Data Generator

File to create: `src/prediction/data_generator.py`

```
PROMPT FOR ANTIGRAVITY / COPILOT:

Write a Python script `data_generator.py` that generates a synthetic
dataset simulating hospital IoMT network telemetry for a time-series
forecasting project. Requirements:

- Use NumPy and Pandas.
- Generate 20,000 rows of telemetry at 1-second intervals for 5 simulated
  devices (ECG-101, ECG-102, O2-201, WEARABLE-301, WEARABLE-302).
- Each row must have these exact fields: timestamp (ISO8601, incrementing
  by 1 second), device_id, bandwidth_mbps (normal range 30-100, mean ~60),
  packet_loss_pct (normal range 0-2%), jitter_ms (normal range 2-15),
  signal_strength_dbm (normal range -50 to -80), current_latency_ms
  (normal range 10-50).
- Inject realistic "degradation events": randomly select ~5% of time
  windows (each lasting 10-60 seconds) where bandwidth_mbps drops
  sharply, packet_loss_pct spikes to 5-20%, jitter_ms rises to 30-100,
  and current_latency_ms spikes to 150-400ms, simulating real network
  congestion. Make the transition into and out of a degradation event
  gradual (ramp up/down), not an instant jump.
- Save the full dataset to data/raw/telemetry_data.csv.
- Add a `--rows` and `--devices` CLI argument (argparse) so the dataset
  size is configurable.
- Include docstrings and inline comments explaining the degradation
  injection logic.
```

3.6 AI Coding Prompt — Preprocessing

File to create: `src/prediction/preprocessing.py`

```
PROMPT FOR ANTIGRAVITY / COPILOT:

Context: I have a CSV at data/raw/telemetry_data.csv with columns:
timestamp, device_id, bandwidth_mbps, packet_loss_pct, jitter_ms,
signal_strength_dbm, current_latency_ms.

Write `preprocessing.py` that:
- Loads the CSV with Pandas, sorts by device_id and timestamp.
- Applies Min-Max scaling (sklearn.preprocessing.MinMaxScaler) to all
  numeric columns, and saves the fitted scaler with joblib to
  data/processed/scaler.pkl (we need this later for inverse-transforming
  predictions).
- Builds sliding-window sequences per device: window_size=20 past
  readings as input (X), and the current_latency_ms of the NEXT reading
  as the prediction target (y). Return X as shape
  (num_samples, window_size, num_features) and y as shape (num_samples,).
```

```
- Splits into train/test sets (80/20), non-shuffled (time-series, so
  preserve order), and saves them as .npy files under data/processed/
  (X_train.npy, y_train.npy, X_test.npy, y_test.npy).
- Wrap all of this in a function `build_dataset(csv_path, window_size=20)`
  that returns X_train, X_test, y_train, y_test, and also runs when the
  file is executed directly via `if __name__ == "__main__":`.
```

3.7 AI Coding Prompt — Model + Training

File to create: `src/prediction/model.py` and `src/prediction/train.py` (intended to run in **Google Colab**)

```
PROMPT FOR ANTIGRAVITY / COPILOT (model.py):

Write `model.py` using TensorFlow/Keras that defines a function
`build_lstm_model(window_size, num_features)` returning a compiled
Keras Sequential model:
- LSTM layer, 64 units, return_sequences=True, input_shape=(window_size,
  num_features)
- Dropout(0.2)
- LSTM layer, 32 units, return_sequences=False
- Dropout(0.2)
- Dense(16, activation='relu')
- Dense(1)  # predicts current_latency_ms
Compile with optimizer='adam', loss='mse', metrics=['mae'].
Add a clear docstring explaining each layer's purpose in plain English
for the report.
```

```
PROMPT FOR ANTIGRAVITY / COPILOT (train.py):

Context: preprocessing.py produces X_train.npy, y_train.npy, X_test.npy,
y_test.npy in data/processed/. model.py exposes build_lstm_model().

Write `train.py` (designed to run in Google Colab) that:
- Loads the .npy files.
- Builds the model via build_lstm_model(window_size=20,
  num_features=<infer from X_train.shape[-1]>).
- Trains for up to 50 epochs with EarlyStopping (patience=5,
  monitor='val_loss'), batch_size=32, validation_split=0.1.
- After training, evaluates on X_test/y_test and prints MAE and RMSE.
- Plots training loss curve AND a "predicted vs actual latency" line
  chart on the test set using matplotlib, saving both figures as PNGs
  to results/graphs/.
- Saves the trained model as model.h5 in the project root's
  src/prediction/ folder, and prints a summary of final metrics in a
  markdown table format we can paste into our report.
```

3.8 AI Coding Prompt — Serving Function (what the rest of the team calls)

File to create: `src/prediction/predict_service.py`

```
PROMPT FOR ANTIGRAVITY / COPILOT:

Context: We have a trained Keras model at src/prediction/model.h5 and a
```

```
fitted MinMaxScaler at data/processed/scaler.pkl. Window size is 20.

Write `predict_service.py` exposing a function:

    def predict_latency(window: list[dict]) -> dict:

Where `window` is a list of exactly 20 telemetry dicts (same schema as
generated by data_generator.py, most recent last). The function must:
- Convert the window into the same numeric feature order used in
  training, scale it with the saved scaler, reshape to
  (1, 20, num_features).
- Run model.predict(), inverse-transform the output back to real
  latency milliseconds.
- Classify network_degradation_risk as "LOW" if predicted_latency_ms <
  75, "MEDIUM" if 75-150, "HIGH" if > 150.
- Return a dict exactly like:
  {"device_id": <from last item in window>, "predicted_latency_ms":
  float, "network_degradation_risk": str, "forecast_horizon_seconds": 30}
- Load the model and scaler once at module import time (not on every
  call) for performance.
- Include a `if __name__ == "__main__":` block that demonstrates calling
  predict_latency() with a dummy window for a quick manual test.
```

3.9 Definition of Done (Charles)

- `model.h5` trained with MAE reported in the write-up
- `predict_latency()` callable and tested standalone
- Two graphs saved: training loss curve, predicted-vs-actual latency
- Research survey (5 papers), research gap, objectives, novelty sections finalized in docs/research/

4. TANMAY — Decision Layer (MCDM / AHP Engine)

4.1 What This Module Does

Takes three inputs — **patient criticality score**, **predicted latency** (from Charles's model), and **current cloud compute cost** — normalizes them, applies an AHP (Analytic Hierarchy Process) weighting matrix, and outputs a routing decision: `"EDGE"` or `"CLOUD"`. Also implements the **emergency override**: if patient criticality crosses a hard threshold, force `"EDGE"` regardless of the AHP score.

4.2 Responsibilities Checklist

- ☐ Implement AHP pairwise comparison matrix and weight derivation
- ☐ Implement criteria normalization (so criticality/latency/cost are comparable)
- ☐ Implement score calculation for "Cloud" and "Edge" paths
- ☐ Implement the comparison + routing decision function
- ☐ Implement the clinical-emergency override rule
- ☐ Expose a clean `router.py` function that the API layer calls
- ☐ Write unit tests with at least 6 varied scenarios (routine/critical/borderline)
- ☐ Produce a routing-distribution chart for the report

4.3 The AHP Method, In Plain Terms (for your own understanding, and for the report)

AHP works in 4 steps: 1. **Define criteria**: Patient Criticality (C1), Predicted Latency (C2), Compute Cost (C3). 2. **Build a pairwise comparison matrix**: how much more important is C1 than C2? C1 than C3? etc. (This produces relative weights, e.g., Criticality=0.55, Latency=0.30, Cost=0.15 — these weights should be justified in the report using your literature survey's emphasis on patient safety first.) 3. **Normalize each criterion's raw value** to a 0–1 scale, so criticality/latency/cost become comparable. 4. **Compute weighted scores** for both the "route to cloud" option and "route to edge" option, and pick whichever scores higher — except when the override rule fires.

4.4 Data Contract (what this module receives and returns)

Input (from API layer, combining Charles's output + patient metadata):

```
{
  "device_id": "ECG-101",
  "patient_criticality_score": 8.5,
  "predicted_latency_ms": 210.5,
  "current_cloud_cost_index": 0.4
}
```

- `patient_criticality_score`: 0–10 scale (0 = routine checkup data, 10 = cardiac arrest alert). This will come from a simple mock/rule (e.g., derived from vitals thresholds) — Rani can help wire this from sensor values, or it can be manually simulated for now.
- `predicted_latency_ms`: from Charles's `predict_latency()`.
- `current_cloud_cost_index`: 0–1 normalized index representing current cloud compute cost/load (can be simulated with a random or time-of-day-based function for now).

Output:

```
{
  "device_id": "ECG-101",
  "routing_decision": "EDGE",
  "reason": "override_critical_patient",
  "cloud_score": 0.42,
  "edge_score": 0.71
}
```

4.5 AI Coding Prompt — AHP Core Logic

File to create: src/decision/ahp.py

PROMPT FOR ANTIGRAVITY / COPILOT:

Write `ahp.py` implementing an Analytic Hierarchy Process (AHP) helper for a 3-criteria decision problem. Requirements:

- A function `get_ahp_weights(pairwise_matrix: list[list[float]]) -> list[float]` that takes a 3x3 pairwise comparison matrix (standard AHP format, e.g. [[1,3,5],[1/3,1,3],[1/5,1/3,1]]) and returns normalized weights for the 3 criteria using the eigenvector method (normalize each column, then average each row) — implement it with plain NumPy, no external AHP library.
- Also implement `consistency_ratio(pairwise_matrix, weights) -> float` that computes the AHP Consistency Ratio (CR) using the standard lambda_max / Random Index method for n=3 (RI=0.58), so we can report in our documentation that our pairwise matrix is consistent (CR < 0.1).
- Define a default pairwise matrix as a module-level constant DEFAULT_PAIRWISE_MATRIX representing: Patient Criticality is moderately more important than Predicted Latency (weight ratio ~3), and much more important than Compute Cost (weight ratio ~5); Predicted Latency is moderately more important than Compute Cost (ratio ~3).
- A function `normalize_criteria(criticality: float, latency_ms: float, cost_index: float) -> tuple[float, float, float]` that scales criticality from its 0-10 range to 0-1, latency_ms from an assumed 0-500ms range to 0-1 (higher latency = higher "urgency to go to edge" signal, so make sure the direction of scaling makes semantic sense), and passes cost_index through unchanged (already 0-1).
- Add docstrings explaining AHP in plain English for a non-expert reader, since this will be referenced in our academic report.
- Include a `if __name__ == "__main__":` demo printing the derived weights and consistency ratio for DEFAULT_PAIRWISE_MATRIX.

4.6 AI Coding Prompt — Router + Override Logic

File to create: src/decision/router.py and src/decision/override_rules.py

PROMPT FOR ANTIGRAVITY / COPILOT (override_rules.py):

Write `override_rules.py` with a single function:

```
    def check_emergency_override(patient_criticality_score: float) -> bool
```

Return True if patient_criticality_score >= 9.0 (representing a "Code Blue" / life-threatening emergency), meaning routing must be forced to EDGE regardless of AHP scoring. Add a docstring explaining the clinical rationale: during true emergencies, we do not want cost-optimization math to add any delay or risk to routing critical alerts locally.

PROMPT FOR ANTIGRAVITY / COPILOT (router.py):

Context: I have ahp.py with get_ahp_weights(), normalize_criteria(), and DEFAULT_PAIRWISE_MATRIX. I have override_rules.py with check_emergency_override(patient_criticality_score).

Write `router.py` exposing:

```
    def route_decision(device_id: str, patient_criticality_score: float,
                       predicted_latency_ms: float,
                       current_cloud_cost_index: float) -> dict
```

Behavior:
1. First call check_emergency_override(). If True, return immediately with routing_decision="EDGE", reason="override_critical_patient", cloud_score=None, edge_score=None.
2. Otherwise: get AHP weights from get_ahp_weights(DEFAULT_PAIRWISE_MATRIX), normalize the 3 input criteria via normalize_criteria().
3. Compute cloud_score as a weighted combination that FAVORS cloud when latency is low and cost is low (i.e., cloud is attractive when the network is healthy and it's cheap); compute edge_score as a weighted combination that FAVORS edge when criticality is high or latency is high (i.e., edge is attractive when the network might fail or the patient is critical). Use the AHP weights to combine these sensibly — comment your formula clearly and explain the reasoning, since we need to defend this math in our viva.
4. Whichever score is higher determines routing_decision ("CLOUD" or "EDGE"), reason="ahp_decision".
5. Return the exact dict shape specified in the "Data Contract" section of our project guide: device_id, routing_decision, reason, cloud_score, edge_score.
6. This function will later become an AWS Lambda handler, so keep all AWS-specific code (like boto3) OUT of this file — it must remain a pure Python function callable locally or wrapped by a Lambda handler later with zero changes to this logic.

This function should be designed so that this exact router.py file can be reused as-is when we later write a thin lambda_handler.py wrapper around it for AWS deployment.

4.7 AI Coding Prompt — Unit Tests

File to create: tests/test_decision.py

PROMPT FOR ANTIGRAVITY / COPILOT:

Context: src/decision/router.py exposes route_decision(device_id, patient_criticality_score, predicted_latency_ms, current_cloud_cost_index) returning a dict with routing_decision, reason, cloud_score, edge_score.

```
Write pytest tests in tests/test_decision.py covering these scenarios:
1. Critical patient (criticality=9.5) with LOW latency and LOW cost ->
   must still route to EDGE (override wins).
2. Routine patient (criticality=1.0), LOW latency, LOW cost -> should
   likely route to CLOUD (assert routing_decision == "CLOUD").
3. Routine patient (criticality=1.0), VERY HIGH latency (450ms), LOW
   cost -> should likely route to EDGE (network is failing).
4. Moderate patient (criticality=5.0), moderate latency (100ms),
   moderate cost (0.5) -> just assert the function returns a valid
   dict with routing_decision in {"EDGE","CLOUD"} (borderline case,
   don't assert direction).
5. Assert that override reason is exactly "override_critical_patient"
   for case 1, and "ahp_decision" for the others.
6. Parametrize at least one test using @pytest.mark.parametrize across
   5 additional criticality values (2,4,6,8,9) to show routing behavior
   trends toward EDGE as criticality increases, and print a small
   summary table of results (for pasting into our report as evidence
   of correct behavior).
```

4.8 Definition of Done (Tanmay)

- `ahp.py`, `override_rules.py`, `router.py` implemented and passing all tests
- Consistency Ratio for the pairwise matrix reported as < 0.1 in the write-up
- Routing distribution chart (e.g., bar chart of Edge vs Cloud decisions across 100 simulated events) saved to `results/graphs/`
- Research survey, research gap, objectives, novelty sections finalized

5. RANI — Execution Layer (Infrastructure, Security, Storage, Dashboard)

5.1 What This Module Does

Takes the routing decision from Tanmay's module and actually **carries it out**: simulates the edge node and cloud node as two separate processes, logs everything, enforces mock authentication (standing in for Cognito/IAM), stores processed data, and surfaces everything on a dashboard (standing in for QuickSight). This is also the layer responsible for the HIPAA/security narrative in the report.

5.2 Responsibilities Checklist

- ☐ Build `edge_node.py` — simulates local hospital edge processing + local alert
- ☐ Build `cloud_node.py` — simulates cloud-side processing + storage
- ☐ Build `auth.py` — mock login/token system standing in for Cognito
- ☐ Build `storage.py` — local persistent storage standing in for S3/DynamoDB
- ☐ Build `logger.py` — structured logging standing in for CloudWatch
- ☐ Build the Streamlit dashboard standing in for QuickSight
- ☐ Write the HIPAA/security narrative for the report (what protections exist and why)
- ☐ Own final end-to-end integration testing (Section 6)

5.3 Data Contract

Input (from the API layer, after Tanmay's router returns a decision):

```
{
  "device_id": "ECG-101",
  "routing_decision": "EDGE",
  "reason": "override_critical_patient",
  "patient_criticality_score": 9.5,
  "predicted_latency_ms": 210.5,
  "timestamp": "2026-09-22T10:15:32Z"
}
```

What Rani's layer does with it: - If `routing_decision == "EDGE"` → call `edge_node.handle_event()` → triggers a local alert (console/log/optional Telegram webhook) and stores the record locally with `storage.py`. - If `routing_decision == "CLOUD"` → call `cloud_node.handle_event()` → stores the record in the "cloud" store (same storage.py, different table/namespace) and marks it for dashboard analytics. - Every event, regardless of path, gets logged via `logger.py` with a timestamp, routing decision, and reason — this log is what the dashboard reads.

5.4 AI Coding Prompt — Storage Layer

File to create: `src/execution/storage.py`

```
PROMPT FOR ANTIGRAVITY / COPILOT:

Write `storage.py` implementing a simple local storage layer using
SQLite (via Python's built-in sqlite3 module) that simulates AWS S3
(cold storage) and DynamoDB (hot storage) with two tables:

- `cold_storage` (simulating S3): columns device_id, timestamp,
  raw_payload (JSON as TEXT), stored_at.
```

- `hot_storage` (simulating DynamoDB): columns device_id, timestamp, routing_decision, reason, predicted_latency_ms, patient_criticality_score, stored_at.

Expose these functions:
- `init_db(db_path="data/processed/app.db")` — creates the tables if they don't exist.
- `save_cold(device_id, timestamp, raw_payload: dict)` — inserts into cold_storage.
- `save_hot(record: dict)` — inserts into hot_storage (record has the same keys as the hot_storage columns above).
- `get_recent_hot_records(limit=100) -> list[dict]` — returns the most recent records from hot_storage as a list of dicts, ordered by stored_at descending, for the dashboard to consume.
- `get_routing_distribution() -> dict` — returns counts like {"EDGE": 42, "CLOUD": 58} by querying hot_storage.

Include clear comments noting "# TODO AWS MIGRATION: replace with boto3 S3 put_object()" and "# TODO AWS MIGRATION: replace with boto3 DynamoDB put_item()" at the relevant functions, so the migration path is obvious later.

5.5 AI Coding Prompt — Auth Layer

File to create: src/execution/auth.py

PROMPT FOR ANTIGRAVITY / COPILOT:

Write `auth.py` implementing a minimal mock authentication layer standing in for AWS Cognito, for a hospital IT admin dashboard:

- A hardcoded dict of demo users: {"admin": "hashed_password"} — use Python's hashlib.sha256 to hash a demo password "hospital123" at module load time (do not store plaintext passwords, even in this mock).
- A function `login(username: str, password: str) -> str | None` that returns a mock JWT-like token (just a random uuid4 string, stored in an in-memory dict mapping token -> username with a 1-hour expiry) if credentials match, else None.
- A function `verify_token(token: str) -> str | None` that returns the username if the token is valid and not expired, else None.
- Add a comment "# TODO AWS MIGRATION: replace with Amazon Cognito user pool authentication" at the top of the file.

5.6 AI Coding Prompt — Logger

File to create: src/execution/logger.py

PROMPT FOR ANTIGRAVITY / COPILOT:

Write `logger.py` using Python's built-in logging module to create a structured logger standing in for Amazon CloudWatch:

- Configure a logger named "telemedicine_system" that writes to both

```
  the console and a rotating file at logs/system.log (use
  logging.handlers.RotatingFileHandler, maxBytes=5_000_000,
  backupCount=3).
- Format: timestamp, log level, message, as JSON lines (use Python's
  json module to serialize each log record's extra fields) so it's easy
  to parse later.
- Expose a function `log_event(event_type: str, details: dict)` that
  logs an INFO-level structured entry, e.g. log_event("ROUTING_DECISION",
  {"device_id": ..., "routing_decision": ..., "reason": ...}).
- Add a comment "# TODO AWS MIGRATION: replace file handler with
  watchtower or boto3 CloudWatch Logs client" at the top.
```

5.7 AI Coding Prompt — Edge Node & Cloud Node

File to create: src/execution/edge_node.py and src/execution/cloud_node.py

```
PROMPT FOR ANTIGRAVITY / COPILOT (edge_node.py):

Context: We have storage.py (save_cold, save_hot) and logger.py
(log_event) already implemented.

Write `edge_node.py` simulating local hospital edge processing,
standing in for AWS IoT Greengrass. Expose:

    def handle_event(event: dict) -> dict

Where `event` contains device_id, routing_decision, reason,
patient_criticality_score, predicted_latency_ms, timestamp, and
(optionally) the raw telemetry payload. The function must:
- Print/log a clear "LOCAL ALERT TRIGGERED" message including
  device_id and patient_criticality_score, simulating paging the
  on-call doctor locally (standing in for Amazon SNS local alert).
- Save the event to hot_storage via storage.save_hot().
- Save the raw payload to cold_storage via storage.save_cold(), if
  present in the event.
- Log the event via logger.log_event("EDGE_PROCESSED", event).
- Return a dict confirming processing: {"status": "processed_at_edge",
  "device_id": ..., "alerted": True}.
- Add a comment "# TODO AWS MIGRATION: this becomes the AWS IoT
  Greengrass local Lambda function" at the top.

PROMPT FOR ANTIGRAVITY / COPILOT (cloud_node.py):

Same context as edge_node.py. Write `cloud_node.py` simulating cloud
processing, standing in for the "route to cloud" execution path.
Expose:

    def handle_event(event: dict) -> dict

The function must:
- NOT trigger any local alert (this is routine data).
- Save the event to hot_storage and cold_storage exactly as
  edge_node.py does.
- Log the event via logger.log_event("CLOUD_PROCESSED", event).
- Return {"status": "processed_in_cloud", "device_id": ...,
```

```
  "alerted": False}.
- Add a comment "# TODO AWS MIGRATION: this becomes an AWS Lambda
  function reading from the main cloud pipeline / API Gateway" at the
  top.
```

5.8 AI Coding Prompt — Dashboard

File to create: `dashboard/app.py`

```
PROMPT FOR ANTIGRAVITY / COPILOT:

Context: storage.py exposes get_recent_hot_records(limit=100) and
get_routing_distribution() reading from data/processed/app.db (SQLite).

Write a Streamlit dashboard `dashboard/app.py`, standing in for Amazon
QuickSight, that:
- Has a simple login form using auth.py's login()/verify_token()
  (username "admin", password "hospital123" for demo).
- After login, shows:
  1. A pie or bar chart of routing_distribution (EDGE vs CLOUD counts)
     using get_routing_distribution().
  2. A live-updating table (st.dataframe) of the most recent 20 events
     from get_recent_hot_records(), showing device_id, timestamp,
     routing_decision, reason, predicted_latency_ms,
     patient_criticality_score.
  3. A line chart of predicted_latency_ms over time for the most recent
     records, to visually show when the system detected degrading
     network conditions.
  4. A metric row (st.metric) showing: total events processed, %
     routed to edge, % routed to cloud.
- Add a manual "Refresh" button (st.button) that re-queries the
  database, since Streamlit doesn't auto-poll by default.
- Keep the layout clean with st.set_page_config(layout="wide") and
  clear section headers.
```

5.9 The HIPAA / Security Narrative (write this into the report, Rani's section)

Even in the local/simulated version, document these protections explicitly — this is what a reviewer will look for regardless of whether real AWS is deployed:

- **Access control**: only authenticated admins (via `auth.py`, standing in for Cognito) can view the dashboard or patient-linked data.
- **Data minimization at the edge**: only metadata needed for routing (criticality score, not full raw payload) needs to cross into the decision layer; raw PHI can stay in cold storage locally when routed to EDGE.
- **Audit trail**: every routing decision is logged immutably via `logger.py` (standing in for CloudWatch), giving a full audit trail for compliance review — this maps directly to the "immutable storage" WORM-compliance idea in the original sequence diagram.
- **Encryption note for the report**: state clearly that in the AWS migration, all data at rest (S3/DynamoDB) will be encrypted via AWS KMS, and all data in transit will use TLS — this satisfies HIPAA's technical safeguard requirements even though local SQLite doesn't encrypt by default.

5.10 Definition of Done (Rani)

- All of `storage.py`, `auth.py`, `logger.py`, `edge_node.py`, `cloud_node.py` implemented and working together
- Dashboard running locally (`streamlit run dashboard/app.py`) and showing live data
- HIPAA/security narrative written for the report
- Research survey, research gap, objectives, novelty sections finalized

6. HOW WE COMBINE EVERYTHING (Integration Layer)

This is the part **everyone must agree on**, since it's the glue between all three modules. One person (suggested: whoever is most comfortable with FastAPI, or split it — Rani leads since she owns execution) builds `src/api/main.py`.

6.1 The Full Call Chain

```
1. New telemetry event arrives (from data_generator.py simulation loop,
   or a live simulated stream)
        |
        v
2. api/main.py calls prediction.predict_service.predict_latency(window)
        -> returns predicted_latency_ms, network_degradation_risk
        |
        v
3. api/main.py calls decision.router.route_decision(
        device_id, patient_criticality_score,
        predicted_latency_ms, current_cloud_cost_index)
        -> returns routing_decision, reason, cloud_score, edge_score
        |
        v
4. api/main.py calls execution.edge_node.handle_event(event)
   OR execution.cloud_node.handle_event(event)
   based on routing_decision
        |
        v
5. execution.logger.log_event(...) records everything
        |
        v
6. dashboard/app.py reads from execution.storage and displays live
```

6.2 AI Coding Prompt — The Integration API

File to create: `src/api/main.py`

```
PROMPT FOR ANTIGRAVITY / COPILOT:

Context: We have three independent modules already built:
- src/prediction/predict_service.py exposing predict_latency(window:
  list[dict]) -> dict with keys device_id, predicted_latency_ms,
  network_degradation_risk, forecast_horizon_seconds.
- src/decision/router.py exposing route_decision(device_id,
  patient_criticality_score, predicted_latency_ms,
  current_cloud_cost_index) -> dict with keys device_id,
  routing_decision, reason, cloud_score, edge_score.
- src/execution/edge_node.py and src/execution/cloud_node.py, both
  exposing handle_event(event: dict) -> dict.
- src/execution/logger.py exposing log_event(event_type, details).

Write a FastAPI app in `src/api/main.py` that:
- Exposes POST /ingest, accepting a JSON body:
  {
    "window": [ ...20 telemetry dicts... ],
```

```
    "patient_criticality_score": float,
    "current_cloud_cost_index": float
  }
- Pipeline inside the endpoint:
  1. Call predict_latency(window) to get predicted_latency_ms and
     network_degradation_risk.
  2. Call route_decision(device_id=window[-1]["device_id"],
     patient_criticality_score, predicted_latency_ms,
     current_cloud_cost_index).
  3. Build a combined `event` dict merging device_id, timestamp
     (use the last telemetry item's timestamp), routing_decision,
     reason, patient_criticality_score, predicted_latency_ms, and the
     raw last telemetry reading as raw_payload.
  4. If routing_decision == "EDGE", call
     execution.edge_node.handle_event(event); else call
     execution.cloud_node.handle_event(event).
  5. Log the full pipeline result via log_event("PIPELINE_COMPLETE",
     event).
  6. Return a JSON response combining the prediction, routing decision,
     and execution result, so a client/tester can see the full
     pipeline's output in one response.
- Add a GET /health endpoint returning {"status": "ok"}.
- Add a GET /stats endpoint that calls
  execution.storage.get_routing_distribution() and returns it directly
  — useful for quick testing without opening the dashboard.
- Use Pydantic models for request validation.
- Include CORS middleware allowing all origins (for local dashboard/
  frontend testing).
- Add a `if __name__ == "__main__":` block that runs the app with
  uvicorn on 0.0.0.0:8000.
```

6.3 AI Coding Prompt — End-to-End Simulation Runner

File to create: src/api/simulate_stream.py

```
PROMPT FOR ANTIGRAVITY / COPILOT:

Context: The FastAPI app in src/api/main.py exposes POST /ingest as
described above (accepts window, patient_criticality_score,
current_cloud_cost_index; returns the full pipeline result).

Write `simulate_stream.py` that:
- Loads data/raw/telemetry_data.csv with Pandas.
- Iterates through it in device-grouped sliding windows of 20 readings
  at a time (reuse the same windowing logic style as preprocessing.py).
- For each window, randomly assigns a patient_criticality_score (mostly
  low values 0-3, but ~5% of windows get a high value 8-10 to simulate
  occasional emergencies) and a current_cloud_cost_index (random 0-1,
  or time-of-day based sine wave for realism).
- POSTs each window to http://localhost:8000/ingest using the
  `requests` library, with a short delay (time.sleep(0.5)) between
  requests to simulate a live stream.
- Prints a one-line summary per request showing device_id,
  predicted_latency_ms, routing_decision.
```

```
- This script is what we'll run live during the demo to show the full
  system working end-to-end.
```

6.4 Integration Testing Checklist (Rani leads, everyone participates)

- ☐ Each person's module works standalone (their own unit tests pass)
- ☐ `feature/prediction`, `feature/decision`, `feature/execution` all merged into `develop` without conflicts
- ☐ `src/api/main.py` successfully imports and calls all three modules
- ☐ Run `uvicorn src.api.main:app --reload` — server starts with no errors
- ☐ Run `simulate_stream.py` — at least 50 events processed end-to-end without crashing
- ☐ Dashboard shows live-updating data while the simulation runs
- ☐ At least one "emergency override" event is visible in the logs/dashboard (confirms Tanmay's override logic actually fires)
- ☐ Full run exported as a screen recording / screenshots for the report and viva

7. AWS Migration Path (For Later — Not Needed Now)

This system is deliberately built so that swapping in real AWS later is a **replacement of implementation, not logic**. Keep this table in the README.

Local Component (now)	Real AWS Service (later)	What changes
`data_generator.py` posting to FastAPI	AWS IoT Core (MQTT ingest)	Replace local POST with MQTT publish to IoT Core topic
`predict_service.py` (local Keras model)	Amazon SageMaker endpoint	Deploy `model.h5` to a SageMaker endpoint; replace local `model.predict()` call with a `boto3` `invoke_endpoint()` call
`router.py` (plain Python function)	AWS Lambda	Wrap `route_decision()` in a `lambda_handler(event, context)` — the internal logic is unchanged
`edge_node.py`	AWS IoT Greengrass local Lambda	Deploy the same handler logic as a Greengrass component
`cloud_node.py`	AWS Lambda (cloud path)	Same handler, deployed as a standard (non-Greengrass) Lambda
`storage.py` (SQLite)	Amazon S3 + DynamoDB	Replace SQLite calls with `boto3` `put_object()` (S3) and `put_item()` (DynamoDB)
`auth.py` (mock tokens)	Amazon Cognito	Replace with Cognito User Pools + hosted UI
`logger.py` (file-based)	Amazon CloudWatch Logs	Replace `RotatingFileHandler` with `watchtower` or `boto3` CloudWatch client
Streamlit dashboard	Amazon QuickSight	Point QuickSight at the DynamoDB/S3 data source instead of SQLite
Local alert print statement	Amazon SNS	Replace print with `boto3` `sns.publish()`

When you're ready to migrate, come back and we'll go service-by-service — each swap is small and isolated because of how the modules are separated now.

8. Full Development Timeline

Phase	What Happens
Setup	Create GitHub repo, branch structure, `requirements.txt`, agree on data schemas in Section 3.3 / 4.4 / 5.3
Module build	Charles, Tanmay, Rani build their modules independently and in parallel, each using the AI prompts above in their own `feature/*` branch
Unit testing	Each person tests their module standalone before opening a PR
Code review	Each PR reviewed by at least one teammate before merging into `develop`
Integration	Build `src/api/main.py` and `simulate_stream.py`, merge everything, run the full pipeline end-to-end
Polish	Fix bugs found during integration, generate all graphs/screenshots needed for the report
Report + Diagrams	Finalize abstract, literature survey, research gaps, objectives, novelty summary, and **redraw the architecture diagrams to match this actual system** (see note below)
Demo prep	Rehearse running `simulate_stream.py` live plus the dashboard for the review panel
AWS migration (optional, later)	Follow Section 7, one service at a time

Important note on diagrams: the existing architecture diagrams (smart-waste-collection sketches, hand-drawn AWS diagrams) need to be redrawn to reflect this actual local-first pipeline plus the target AWS architecture side by side — this guide's Section 2.1 diagram is the basis for that.

9. Requirements File

File to create: `requirements.txt`

```
fastapi
uvicorn[standard]
pydantic
numpy
pandas
scikit-learn
tensorflow
matplotlib
joblib
streamlit
requests
pytest
python-multipart
```

10. Final Pre-Submission Checklist

- ☐ All three modules merged into `develop`, then `main`, tagged `v1.0-phase1`
- ☐ End-to-end simulation runs cleanly for at least 100 events
- ☐ All graphs generated: training loss, predicted-vs-actual latency, routing distribution
- ☐ Dashboard demo-ready
- ☐ Architecture diagrams redrawn to match the actual local + target-AWS system
- ☐ Contribution matrix fully filled in (not just Literature Survey / Research Gap rows)
- ☐ HIPAA/security narrative written into the report
- ☐ AWS migration table (Section 7) included in the report as evidence of cloud-architecture understanding
- ☐ README.md includes: project overview, repo structure, how to run locally, migration plan

---

Questions during development? Bring back this guide and the specific module/section number — it'll be faster to debug or extend a specific piece than to re-explain the whole system each time.